Kalasalingam Academy of Research and education

Podcast and YouTube Script Planner

Project ID: PRJ045

Team ID: Team Cr7

Team GitHub Repository: https://github.com/rajiniarepalli/Podcast-and-YouTube-Script-planner

Student Name	Registration Number	Individual GitHub Link
Arepalli Rajini	99230040480	https://github.com/rajiniarepalli/Podcast-and-YouTube-Script-planner
Varugu Md Anees	99230040448	https://github.com/anees-codecut/GenAi
Annavaram Likiteswar reddy	99230040475	https://github.com/likiteswar0479-design/GenAI.git

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction 1

2. Problem Statement & Objectives Section

3. System Architecture (RAG Pipeline) Section

4. Implementation Details & Tech Stack Section

5. Results and Testing Section

6. Conclusion & Future Scope Section6

1. Introduction

Creating a podcast episode or a YouTube video starts with a script, and writing a good one is slow. Creators must research the topic, structure the content, write a strong hook, keep the tone consistent and tailor everything to the platform and audience. Large Language Models (LLMs) can speed this up, but a plain chat prompt gives no structure, no reusable settings and no easy way to refine the result.

2 Problem Statement & Objectives

Problem: Content creators spend a large share of their time drafting, restructuring and rewriting scripts. Beginners struggle with structure (intro, main segments, call-to-action), while experienced creators lose time on repetitive drafting. General-purpose AI chat tools need carefully written prompts each time and do not offer a workflow built for podcasts and videos.

Objectives:

- To build a web application that generates podcast and YouTube scripts from simple user inputs.
- To let users customize the output (for example topic, platform, tone and length) instead of accepting a fixed result.
- To allow users to refine generated scripts so that the first draft can be improved iteratively.
- To integrate the Google Gemini API through a FastAPI backend that keeps API access on the server side.
- To provide a clean, responsive React.js interface and deploy it publicly for easy access.

3 System Architecture (RAG Pipeline)

The application follows a client-server architecture with three layers: the React.js frontend, the FastAPI backend and the Google Gemini API. This matches the two top-level folders of the repository, frontend and backend

3.1 Frontend (Presentation Layer)

The React.js interface collects the user's input, sends it to the backend and displays the generated script. It is hosted on Vercel and is available as the public live application.

3.2 Backend (Application Layer)

The FastAPI service, written in Python, receives requests from the frontend, builds a structured prompt from the user's inputs, calls the Gemini API and returns the generated text. Keeping this logic on the server protects the API key and keeps prompt design in one place.

3.3 AI Layer

The Google Gemini API generates the script content. Refinement requests send the existing script together with the user's change instructions so that the model can revise the draft.

3.4 Request flow

Step	Description
1	User enters script details in the React interface
2	Frontend sends the request to the FastAPI backend.
3	Backend builds a prompt and calls the Google Gemini API.
4	Gemini returns the generated script
5	Backend returns the response; frontend displays it for review
6	User customizes or refines the script and the cycle repeats

4 Implementation Details & Tech Stack

Component	Technology Used	Purpose
Frontend	React.js	User interface for entering inputs and viewing/refining scripts
Backend	Python, FastAPI	REST API, prompt construction and communication with Gemini
AI Model	Google Gemini API	Generating and refining script content
Hosting	Vercel	Public deployment of the web application
Version Control	Git and GitHub	Team collaboration and source management

4.1 Repository Structure

```
Podcast-and-YouTube-Script-planner/
  backend/      (Python / FastAPI service)
  frontend/     (React.js application)
  setup.py
  .gitignore
```

4.2 Key Features

- Script generation: produces a complete podcast or YouTube script from the user's inputs.
- Customization: the user controls the details of the script that is generated.
- Refinement: existing scripts can be improved without starting over.
- Secure API usage: Gemini is called from the backend so credentials are not exposed in the browser.
- Live deployment: accessible from any browser at the public Vercel link

5 Results and Testing

The application was deployed and made publicly available as ScriptCraft AI. Testing was performed by entering a variety of topics and settings through the interface and checking that the backend returned a coherent, well-structured script each time, and that the refinement option correctly revised an existing script. The generated scripts reduce the time needed to produce a first draft compared with writing from scratch, leaving the creator free to focus on editing and delivery. Note: add your own measured results here (for example the number of test topics used and average response time) before submission.

6 Conclusion & FutureScope

Conclusion: The project successfully combines a React.js frontend, a FastAPI backend and the Google Gemini API into a practical tool for content creators. It turns a few inputs into a usable script and supports further refinement, saving time while improving the quality and consistency of podcast and YouTube content.
Future Scope:
User accounts so that scripts can be saved, organized and revisited.
Ready-made templates for different formats such as interviews, solo episodes, tutorials and shorts.
Export options (PDF, DOCX, plain text) for generated scripts.
Additional tools such as title, description and thumbnail-idea suggestions, and SEO keyword support.
Multi-language script generation and a feedback mechanism to keep improving output quality.